JSTL (Java Standard Tag Library)

JSTL provides an SQL tag library that allows a JSP page to perform database-related operations without writing JDBC code directly in the JSP page. It is mainly useful for educational purposes and small JSP applications. In larger applications, database logic is normally placed in DAO/service classes rather than directly in JSP.

1. JSTL SQL Tag Library

The JSTL SQL library provides tags for:

- Creating a database connection
- Executing SELECT queries
- Executing INSERT, UPDATE, and DELETE statements
- Using parameters in SQL queries
- Executing prepared statements
- Performing transactions

The SQL tag library URI is:

http://java.sun.com/jsp/jstl/sql

The prefix is generally:

SQL

Therefore, the taglib declaration is:

<%@ taglib prefix="sql" uri="http://java.sun.com/jsp/jstl/sql" %>

2. Important JSTL SQL Tags

Tag	Purpose
<sql:setDataSource>	Creates/configures a database data source
<sql:query>	Executes a SELECT query
<sql:update>	Executes INSERT, UPDATE, or DELETE
<sql:param>	Supplies parameter values
<sql:dateParam>	Supplies date/time parameter values
<sql:transaction>	Groups multiple database operations into a transaction

3. <sql:setDataSource> Tag

The <sql:setDataSource> tag is used to configure a database connection.

Syntax

```
<sql:setDataSource
   var="db"
   driver="com.mysql.cj.jdbc.Driver"
   url="jdbc:mysql://localhost:3306/college"
   user="root"
   password="" />
```

Important attributes

Attribute	Description
var	Name used to refer to the data source
driver	JDBC driver class
url	Database connection URL
user	Database username
password	Database password
scope	Scope of the data source

For MySQL:

driver="com.mysql.cj.jdbc.Driver"

Example:

url="jdbc:mysql://localhost:3306/college"

4. <sql:query> Tag

The <sql:query> tag executes a SELECT SQL statement.

Syntax

```
<sql:query var="result" dataSource="${db}">
   SELECT * FROM student
</sql:query>
```

Here:

- result stores the query result.
- db refers to the configured data source.
- ${db} is an EL expression.

The returned result contains rows and columns that can be accessed using JSTL/EL.

5. Displaying Query Results

Suppose the database contains:

```
student
---------------------------------
id   name      city
---------------------------------
1    Amit      Ahmedabad
2    Ravi      Rajkot
3    Neha      Surat
```

The query can be:

```
<sql:query var="result" dataSource="${db}">
   SELECT * FROM student
</sql:query>
```

The rows can then be displayed using JSTL:

```
<c:forEach var="row" items="${result.rows}">
   ${row.id}
   ${row.name}
   ${row.city}
</c:forEach>
```

This requires the JSTL Core library:

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

6. SQL Formatting Using HTML Table

A common JSP application is to retrieve database records and display them in a properly formatted HTML table.

Example:

```
<table border="1">
   <tr>
      <th>ID</th>
      <th>Name</th>
      <th>City</th>
   </tr>

   <c:forEach var="row" items="${result.rows}">
      <tr>
         <td>${row.id}</td>
         <td>${row.name}</td>
         <td>${row.city}</td>
```

```
        </tr>
    </c:forEach>
</table>
```

This produces a structured table.

7. <sql:update> Tag

The <sql:update> tag is used for:

- INSERT
- UPDATE
- DELETE

INSERT Example

```
<sql:update dataSource="${db}">
    INSERT INTO student VALUES (4, 'Kiran', 'Vadodara')
</sql:update>
```

UPDATE Example

```
<sql:update dataSource="${db}">
    UPDATE student SET city='Ahmedabad' WHERE id=4
</sql:update>
```

DELETE Example

```
<sql:update dataSource="${db}">
    DELETE FROM student WHERE id=4
</sql:update>
```

8. <sql:param> Tag

<sql:param> is used to provide parameter values to an SQL statement.

It is preferable to using string concatenation because it allows the SQL operation to use a prepared statement internally.

Example:

```
<sql:update dataSource="${db}"
          sql="INSERT INTO student(id,name,city) VALUES (?, ?, ?)">

    <sql:param value="${param.id}" />
    <sql:param value="${param.name}" />
    <sql:param value="${param.city}" />
```

</sql:update>

Here:

?

represents a parameter placeholder.

The values are supplied in the same order:

First ? → id
Second ? → name
Third ? → city

9. <sql:query> with Parameters

Example:

```
<sql:query var="result" dataSource="${db}"
        sql="SELECT * FROM student WHERE city=?">

   <sql:param value="${param.city}" />

</sql:query>
```

If the URL is:

students.jsp?city=Ahmedabad

then:

${param.city}

contains:

Ahmedabad

The resulting SQL operation searches for students from Ahmedabad.

10. Complete JSTL SQL Program

Program: Display Student Records from MySQL

Database

Create database:

CREATE DATABASE college;

Select database:

USE college;

Create table:

```
CREATE TABLE student
(
   id INT PRIMARY KEY,
   name VARCHAR(50),
   city VARCHAR(50)
);
```

Insert records:

```
INSERT INTO student VALUES
(1, 'Amit', 'Ahmedabad'),
(2, 'Ravi', 'Rajkot'),
(3, 'Neha', 'Surat'),
(4, 'Kiran', 'Vadodara');
```

JSP Program

student.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>

<%@ taglib prefix="sql" uri="http://java.sun.com/jsp/jstl/sql" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!DOCTYPE html>
<html>
<head>
   <title>Student Records</title>

   <style>
      table {
         border-collapse: collapse;
         width: 600px;
      }

      th, td {
         border: 1px solid black;
         padding: 10px;
         text-align: center;
      }
```

```
        th {
            background-color: lightgray;
        }
    </style>
</head>

<body>

<h2>Student Records</h2>

<sql:setDataSource
    var="db"
    driver="com.mysql.cj.jdbc.Driver"
    url="jdbc:mysql://localhost:3306/college"
    user="root"
    password="" />

<sql:query var="result" dataSource="${db}">
    SELECT id, name, city
    FROM student
    ORDER BY id
</sql:query>

<table>

    <tr>
        <th>Student ID</th>
        <th>Student Name</th>
        <th>City</th>
    </tr>

    <c:forEach var="row" items="${result.rows}">

        <tr>
            <td>${row.id}</td>
            <td>${row.name}</td>
            <td>${row.city}</td>
        </tr>

    </c:forEach>

</table>

</body>
</html>
```

Working

The program works in the following sequence:

JSP Page
  ↓
sql:setDataSource
  ↓
MySQL Database
  ↓
sql:query
  ↓
result object
  ↓
result.rows
  ↓
c:forEach
  ↓
HTML Table

11. JSTL SQL Program with Search

A more useful example is searching students by city.

HTML Form

```
<form method="get">

   Enter City:

   <input type="text" name="city">

   <input type="submit" value="Search">

</form>
```

SQL Query

```
<sql:query var="result"
          dataSource="${db}"
          sql="SELECT * FROM student WHERE city=?">

   <sql:param value="${param.city}" />

</sql:query>
```

Display

```
<table border="1">

   <tr>
      <th>ID</th>
      <th>Name</th>
```

```
      <th>City</th>
   </tr>

   <c:forEach var="row" items="${result.rows}">

      <tr>
         <td>${row.id}</td>
         <td>${row.name}</td>
         <td>${row.city}</td>
      </tr>

   </c:forEach>

</table>
```

12. JSTL SQL INSERT Program

The following example accepts student information from an HTML form and inserts it into MySQL.

```
<%@ page contentType="text/html;charset=UTF-8" %>

<%@ taglib prefix="sql" uri="http://java.sun.com/jsp/jstl/sql" %>

<html>
<head>
   <title>Insert Student</title>
</head>

<body>

<h2>Student Registration</h2>

<form method="post">

   Student ID:
   <input type="text" name="id">
   <br><br>

   Student Name:
   <input type="text" name="name">
   <br><br>

   City:
   <input type="text" name="city">
   <br><br>

   <input type="submit" value="Insert Student">
```

```
</form>

<sql:setDataSource
   var="db"
   driver="com.mysql.cj.jdbc.Driver"
   url="jdbc:mysql://localhost:3306/college"
   user="root"
   password="" />

<%
   if ("POST".equalsIgnoreCase(request.getMethod())) {
%>

<sql:update dataSource="${db}"
         sql="INSERT INTO student(id,name,city) VALUES (?, ?, ?)">

   <sql:param value="${param.id}" />
   <sql:param value="${param.name}" />
   <sql:param value="${param.city}" />

</sql:update>

<p>Student record inserted successfully.</p>

<%
   }
%>

</body>
</html>
```

13. JSTL SQL UPDATE Program

```
<sql:update dataSource="${db}"
         sql="UPDATE student SET name=?, city=? WHERE id=?">

   <sql:param value="${param.name}" />
   <sql:param value="${param.city}" />
   <sql:param value="${param.id}" />

</sql:update>
```

The three parameters correspond to:

First ? → student name
Second ? → city
Third ? → student ID

14. JSTL SQL DELETE Program

```
<sql:update dataSource="${db}"
        sql="DELETE FROM student WHERE id=?">

   <sql:param value="${param.id}" />

</sql:update>
```

The value of:

${param.id}

is supplied to the SQL statement.

15. JSTL SQL Transaction

A transaction is useful when multiple database operations must be treated as a single unit.

The basic structure is:

```
<sql:transaction dataSource="${db}">

   <sql:update>
      INSERT INTO student VALUES (10, 'Raj', 'Rajkot')
   </sql:update>

   <sql:update>
      UPDATE student SET city='Ahmedabad' WHERE id=2
   </sql:update>

</sql:transaction>
```

If an error occurs during the transaction, the database operations can be rolled back according to the transaction behavior.

16. Important JSTL SQL + EL Relationship

JSTL SQL and EL are commonly used together.

For example:

```
<sql:query var="result" dataSource="${db}">
   SELECT * FROM student
</sql:query>
```

The query result can be accessed through EL:

${result}

Rows:

${result.rows}

Individual column values:

${row.id}
${row.name}
${row.city}

Request parameters:

${param.id}
${param.name}
${param.city}

This combination makes JSP database pages considerably shorter than writing raw JDBC code inside scriptlets.

17. JSTL SQL Tags — Quick Revision

Tag	Main Function	Example
sql:setDataSource	Configure database	<sql:setDataSource ...>
sql:query	Execute SELECT	<sql:query ...>
sql:update	INSERT/UPDATE/DELETE	<sql:update ...>
sql:param	Set ? parameter	<sql:param value="..."/>
sql:dateParam	Set date parameter	<sql:dateParam .../>
sql:transaction	Transaction management	<sql:transaction ...>

In JSP with JSTL, the Core (c) and Formatting (fmt) tag libraries are two of the most commonly used JSTL libraries.

1. JSTL Core Tag Library

The Core library provides tags for common programming operations in JSP such as:

- Variables
- Conditions
- Loops
- URL handling
- Output
- Removing variables
- Exception handling

The Jakarta JSTL Core taglib directive is:

<%@ taglib prefix="c" uri="jakarta.tags.core" %>

Here, c is the prefix used to access Core tags.

For example:

<c:out value="${name}" />

Important JSTL Core Tags

Tag	Purpose
<c:out>	Display a value
<c:set>	Create/set a variable
<c:remove>	Remove a variable
<c:if>	Conditional statement
<c:choose>	Multiple conditions
<c:when>	Condition inside choose
<c:otherwise>	Default condition
<c:forEach>	Loop
<c:forTokens>	Loop through tokens
<c:url>	Create URL
<c:param>	Add URL parameter
<c:redirect>	Redirect to another page
<c:catch>	Handle exceptions
<c:import>	Import another resource

2. <c:out> — Display Data

<c:out value="Hello JSP" />

Output:

Hello JSP

Using an EL variable:

<c:out value="${name}" />

If:

```
<%
   request.setAttribute("name", "Chirag");
%>
```

then:

Chirag

is displayed.

3. <c:set> — Create Variable

<c:set var="name" value="Chirag" />

Name: <c:out value="${name}" />

Output:

Name: Chirag

You can also perform calculations:

```
<c:set var="a" value="10" />
<c:set var="b" value="20" />

<c:out value="${a + b}" />
```

Output:

30

4. <c:if> — Conditional Statement

```
<c:set var="age" value="20" />

<c:if test="${age >= 18}">
   You are eligible to vote.
</c:if>
```

Output:

You are eligible to vote.

This is similar to:

```
if(age >= 18) {
   ...
}
```

5. <c:choose> — Multiple Conditions

```
<c:set var="marks" value="75" />

<c:choose>

   <c:when test="${marks >= 70}">
      Grade A
   </c:when>

   <c:when test="${marks >= 50}">
      Grade B
   </c:when>

   <c:when test="${marks >= 35}">
      Grade C
   </c:when>

   <c:otherwise>
      Fail
   </c:otherwise>

</c:choose>
```

This is similar to:

```
if
else if
else
```

6. <c:forEach> — Loop

```
<c:forEach var="i" begin="1" end="5">

   Number: <c:out value="${i}" />
   <br>

</c:forEach>
```

Output:

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

It is similar to:

```
for(int i = 1; i <= 5; i++)
```

7. JSTL Formatting (fmt) Library

The Formatting library is used for:

- Number formatting
- Currency formatting
- Date and time formatting
- Parsing numbers
- Parsing dates
- Locale management
- Internationalization

The Jakarta JSTL Formatting taglib directive is:

```
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
```

Here:

fmt

is the prefix.

Important fmt Tags

Tag	Purpose
<fmt:formatNumber>	Format numbers
<fmt:parseNumber>	Convert formatted text to number
<fmt:formatDate>	Format date/time
<fmt:parseDate>	Convert text to date
<fmt:setLocale>	Set locale
<fmt:setBundle>	Set resource bundle
<fmt:message>	Display localized message
<fmt:requestEncoding>	Set request encoding

8. <fmt:formatNumber>

Suppose:

<c:set var="price" value="12500.50" />

You can format it:

<fmt:formatNumber value="${price}" />

Output may be:

12,500.50

Currency Formatting

```
<fmt:formatNumber
   value="${price}"
   type="currency"
/>
```

Depending on the locale, it may display a currency symbol and formatted amount.

For Indian currency, you can explicitly set the locale:

<fmt:setLocale value="en_IN" />

```
<fmt:formatNumber
   value="${price}"
   type="currency"
/>
```

This is useful for displaying prices in an Indian application.

9. Number Formatting

You can specify decimal places:

```
<fmt:formatNumber
   value="125.6789"
   pattern="#,##0.00"
/>
```

Output:

125.68

Another example:

```
<fmt:formatNumber
   value="1234567.89"
   pattern="#,##0.00"
/>
```

Output:

1,234,567.89

10. Percentage Formatting

```
<fmt:formatNumber
   value="0.75"
   type="percent"
/>
```

Output:

75%

This is useful for displaying:

- Percentage
- Discount
- Tax rate
- Result percentage

11. <fmt:formatDate>

Suppose a Java Date object is available:

```
<fmt:formatDate
```

```
   value="${today}"
   pattern="dd-MM-yyyy"
/>
```

Output:

06-10-2026

You can also display time:

```
<fmt:formatDate
   value="${today}"
   pattern="dd-MM-yyyy HH:mm:ss"
/>
```

Output:

06-10-2026 14:30:25

12. Complete Core + Formatting Example

Here is a small JSP program using both libraries:

```
<%@ page contentType="text/html;charset=UTF-8" %>

<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>

<html>
<head>
   <title>JSTL Core and Formatting</title>
</head>

<body>

<h2>JSTL Core and Formatting Example</h2>

<c:set var="name" value="Chirag" />
<c:set var="marks" value="78.567" />
<c:set var="salary" value="45000.50" />

<h3>Core Tags</h3>

Name:
<c:out value="${name}" />

<br><br>

Marks:
```

```
<c:out value="${marks}" />

<br><br>

<c:if test="${marks >= 35}">
   Result: Pass
</c:if>

<br><br>

<c:choose>

   <c:when test="${marks >= 70}">
      Grade: A
   </c:when>

   <c:when test="${marks >= 50}">
      Grade: B
   </c:when>

   <c:otherwise>
      Grade: C
   </c:otherwise>

</c:choose>

<h3>Formatting Tags</h3>

Original Marks:
<c:out value="${marks}" />

<br><br>

Formatted Marks:

<fmt:formatNumber
   value="${marks}"
   pattern="#,##0.00"
/>

<br><br>

Salary:

<fmt:formatNumber
   value="${salary}"
   type="currency"
/>

<br><br>
```

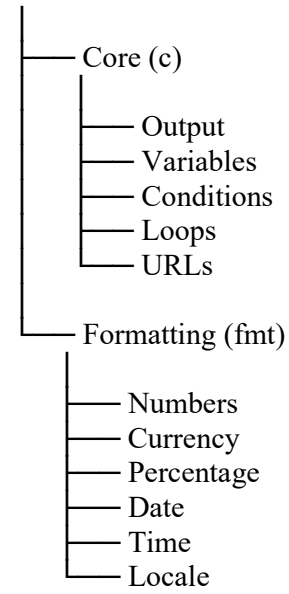
Percentage:

```
<fmt:formatNumber
   value="0.785"
   type="percent"
/>

</body>
</html>
```

Core vs Formatting

The easiest way to remember the difference is:



In one sentence

c is mainly for controlling JSP page logic, while fmt is mainly for formatting numbers, dates, currency, percentages, and handling localization.

For Tomcat 10/11 with Jakarta JSTL, use:

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
```

These are different from the older JSTL URIs such as http://java.sun.com/jsp/jstl/core, which are commonly seen in older Java EE/Tomcat examples.